

Empyralis Agent Workspace Design

Executive Summary (One Page)

Empyralis should ship a transparent, safe and scalable multi-agent workspace for normal users and businesses. **Centralized orchestration** is critical: an **orchestrator** receives tasks, routes them to specialist agents, monitors progress and enforces guardrails. The platform must minimize complexity and only adopt multi-agent workflows when single-agent solutions are insufficient; this aligns with Microsoft guidance that multi-agent orchestration adds coordination overhead and must be justified ¹. Agents will run inside **worker pools** managed by the orchestrator; workers can be added or removed based on workload and cost thresholds. **Approval gates** and safety guardrails protect against risky actions. A unified **Agent Workspace** UI must surface task queues, agent state, tools used, files touched, and pending approvals to give operators full visibility. Key metrics include queue length, SLA adherence, cost, escalation rate and business outcomes. The Minimal Viable Product (MVP) should focus on Shopify/e-commerce workflows and a personal-assistant scenario, shipping within two weeks, while more advanced collaboration and analytics can wait. By prioritizing transparency, guardrails and tight integration with business tools, Empyralis can deliver a product that earns user trust and demonstrates product-market fit.

Detailed Specification

1. Architecture

Components:

1. **Orchestrator:** A central controller that accepts tasks via API/UI, assigns them to appropriate agents, tracks state and enforces policies. It maintains a task queue with status (queued, running, awaiting approval, completed) and computes an ETA. The orchestrator chooses the simplest orchestration pattern that meets the task's needs; Microsoft recommends starting with single agents and moving to multi-agent only when necessary ¹.
2. **Worker Pools:** Pools of execution environments (e.g., containers) for running agents. Each pool hosts agents of a similar risk tier. The orchestrator dispatches tasks to pools and monitors CPU, memory and token usage. GraphBit's concurrent orchestration shows that running tasks in parallel improves efficiency ², but resource consumption must be managed and may require memory management ³.
3. **Agents:** Independent modules with a clear role. Each agent uses an LLM or other model plus a set of tools (e.g., APIs, connectors). Agents follow **narrow, scoped capabilities**—guardrails guidance warns against “god tools” ⁴. Agents communicate with the orchestrator via structured messages (task description, context, outputs). They do not call each other directly; all hand-offs go through the orchestrator.
4. **Approval Gate:** A service that pauses execution when tasks cross risk thresholds. Guardrails patterns recommend tiered execution modes—log-only, auto for low-risk, and human approval for risky actions ⁵. Approval tasks appear in a review queue in the UI; operators can approve, modify or reject with full context.

5. **Memory Store:** Shared memory (vector database + key-value store) to persist task context, intermediate results and RAG sources. Memory retrieval should be guided by retrieval and reasoning patterns to improve accuracy before raising autonomy ⁶.
6. **Connector Layer:** A library of signed connectors (database, Shopify, email, calendar, spreadsheets, messaging). Each connector declares scopes and limits; recommended practice is to mandate signed tool adapters to reduce supply-chain risk ⁷. Connectors enforce server-side policy limits (e.g., refund limit) so that even if the agent misbehaves, the call fails safely ⁸.
7. **Observability & Audit Plane:** Logging proxies and monitors record prompts, chain-of-thought (if permitted), tool calls, external actions, resource usage and outcomes. Transparent systems must log 100% of prompts and actions ⁹ and provide metrics like cost anomalies and escalation rates ¹⁰.

2. Agent Role Taxonomy

Define a **minimum set of specialist agents** to avoid overlap and cognitive load. Roles follow the **single-responsibility principle**:

Role	Description & Responsibilities	Notes
Planner/ Coordinator	Break down a user goal into ordered tasks, select appropriate agents/tools, manage dependencies and compile results. Uses retrieval to ground plans in policies and docs, then delegates tasks to specialist agents.	Only high-level planning; does not execute external actions.
Researcher	Performs web and document retrieval, summarization and extraction. Uses safe browsing and citation capture.	Returns structured summaries; does not call external business APIs.
Data Analyst	Operates on spreadsheets/dataframes, performs calculations, pivot tables and charting.	Integrates with spreadsheets connector.
Messaging Agent	Drafts and sends emails/messages using templates and CRM data; schedules calendar events.	Low-risk actions; auto-approved below thresholds.
Shopify Agent	Handles e-commerce operations: check inventory, update product listings, process orders, issue refunds under policy limits.	High-risk tier: refunds and order modifications require approval above thresholds ¹¹ .
Finance Agent	Performs accounting queries, small expense reimbursements, generates invoices and summaries.	All payments above threshold require two-key approval ¹² .
Personal Assistant Agent	Coordinates tasks like travel bookings, reminders, and to-dos; interacts with messaging, calendar and search agents.	Emphasize user preferences and time zones.
Quality & Safety Critic	Runs evaluation checks on agent outputs using a separate model; validates JSON schemas, business rules and content moderation ¹³ .	Always runs before external actions; flags anomalies to the orchestrator.

Role	Description & Responsibilities	Notes
DevOps Agent (Phase 2)	For technical users; deploys code, monitors infrastructure, restarts services.	Sandboxed; high-risk actions require multi-approval.

3. Auto-scaling Policy

Empyralis must handle varying workloads while controlling costs. A **scaling controller** monitors the task queue and resource metrics and spawns or stops workers accordingly:

1. **Spawn rules:**

2. If average queue wait time > target SLA (e.g., 60 s) and CPU utilization < 80%, spawn additional worker(s) in the relevant pool.
3. When tasks require concurrent orchestration (fan-out), allocate separate workers for each branch, but limit concurrency to avoid resource exhaustion—Microsoft notes that concurrent orchestration reduces latency but can increase resource use and complexity ¹⁴.
4. Create workers only within pre-defined quotas per role and risk tier (e.g., max 5 concurrent Shopify agents, max 2 finance agents) to enforce policy.

5. **Stop rules:**

6. When queue length < threshold and workers are idle for a cooldown period (e.g., 5 minutes), scale down the pool to a minimum baseline (e.g., 1 worker per role).
7. Decommission workers gracefully after finishing current tasks and persisting state.

8. **Limits and cooldown:**

9. Hard caps on total workers across all pools to prevent cost overruns; budgets can be enforced by the connector layer's rate limits and cost monitoring ⁵.
10. Minimum cooldown between scale events (e.g., 2 minutes) to avoid thrashing.

11. **SLA triggers:**

12. If tasks exceed the SLA for critical workflows (e.g., Shopify order processing must complete within 5 minutes), the orchestrator prioritizes these tasks and may temporarily deprioritize lower-risk tasks.
13. Escalation rate (percentage of tasks requiring human intervention) and error rate feed into scaling decisions; high error rates may prompt more critic agents rather than more workers.

4. Inter-agent Protocol (Task Handoff Contract)

To avoid uncontrolled agent-to-agent chatter, tasks are passed through the orchestrator using a structured protocol:

- **Task Packet:** contains `task_id`, `role`, `input_context`, `desired_output_schema`, `tools_allowed`, `risk_tier`, and `deadline`. Agents must respond with `task_id`, `output` (structured), `actions_proposed` (for approval), and `confidence`.

- **State Isolation:** Agents cannot access each other's memory directly; they reference shared memory via the orchestrator's API. This prevents state corruption and ensures policies apply.
- **Retry & Timeout:** The orchestrator imposes a maximum number of iterations per task; if an agent fails or loops, the orchestrator aborts and escalates. Microsoft recommends setting iteration limits to guard against infinite loops ¹⁵.
- **Error Handling:** Agents must classify errors as transient (retryable) or fatal. Fatal errors trigger fallback strategies or human intervention.
- **Versioning & Audit:** Each task includes metadata about agent version, prompt and tool versions; the observability plane logs all messages ¹⁶ for auditing.

5. Agent Workspace UI Specification

A unified web app provides transparency and control. Required elements:

1. **Task Queue Panel:** Table showing pending/running/completed tasks. Columns: Task ID, User, Goal, Assigned Agent, State, Risk Tier, ETA, Cost, Escalation Needed. Operators can sort/filter by state or risk. Clicking a row opens the task detail.
2. **Task Detail Panel:** Shows the conversation between the agent and orchestrator, sanitized chain-of-thought, tool calls, retrieved documents, files touched, and memory reads/writes. Each proposed external action (e.g., refund, email send) is listed with a **one-click Approve/Edit/Reject** control. This satisfies the pattern of "propose and approve" for medium and high-risk actions ¹⁷.
3. **Approval Queue:** List of actions awaiting human approval. Columns: Action, Agent, Context Summary, Risk Level, Expires At. Reviewers can bulk approve or reject.
4. **Agent Status Panel:** Displays active agents and worker pools with metrics (CPU usage, tokens consumed, memory footprint, number of tasks processed). Operators can pause, resume or terminate agents and adjust pool size.
5. **Logs & Audit Panel:** Searchable log of prompts, intermediate reasoning, tool calls and external API calls, with filters by agent, user or time. Offers export for compliance.
6. **Settings & Policies Panel:** Manage policies (risk thresholds, refund limits, role permissions), connectors and tool scopes. Each connector shows its signed manifest, scopes and usage.
7. **KPI Dashboard:** Visualizes operator and business KPIs. Metrics include queue length, average response time, escalation rate ¹⁸, consistency rate, cost anomalies, and business outcomes (e.g., orders processed, revenue). Charts track trending metrics and anomalies.

Transparency Model:

- **Full visibility with abstraction:** Show enough reasoning (structured plans, high-level justification) without exposing sensitive prompt engineering. Expose all tool calls, retrieved documents and external actions. Include risk tier classification and policy references.
- **Audit Trail:** Every prompt, intermediate state and action has a permanent record ¹⁶. Operators can trace a result back to prompts and tool calls.
- **Human override:** Operators can cancel tasks, override agent decisions, or modify outputs before execution.

6. Development Scope

MVP – 2-week Scope (P0/P1)

- **Core architecture:** Implement orchestrator, simple worker pool, memory store and connector layer with Shopify, email, calendar and spreadsheet connectors.
- **Agent roles:** Planner, Researcher, Messaging, Shopify Agent and Quality & Safety Critic.

- **UI basics:** Task queue panel, task detail view with action approvals, agent status panel, and basic logs.
- **Safety & policies:** Input filtering (PII redaction, prompt injection detection) and output schema validation ¹⁹ ²⁰ . Execution modes (log-only vs auto vs approval) per tool ⁵ .
- **Auto-scaling:** Simple scaling based on queue length and idle time with hard cap.
- **Shopify workflow integration:** Use Shopify connector to fetch orders, update inventory, issue small refunds under policy; human approval for refunds >\$100.
- **Personal assistant workflow:** Schedule meetings (calendar), draft emails (messaging), and summarise research (researcher). Emphasize time-zone awareness for Asia/Jakarta.

Phase 2 – 6-week Scope (P2)

- **Additional agents:** Finance Agent, Data Analyst, DevOps Agent (technical users). Add Group Chat orchestration pattern for collaborative brainstorming and voting when tasks benefit from multiple perspectives ²¹ .
- **Advanced scaling controller:** SLA-based scaling with predictive models, dynamic risk-tier quotas and GPU usage optimisation.
- **Enhanced observability:** Integrate an AI gateway/proxy to log all prompts and responses ²² ; implement reliability scoring metrics (consistency, escalation, exploit exposure) ¹⁸ .
- **Roles & permission management:** UI for assigning per-tool permissions to agents and mapping them to users.
- **Template library & workflow builder:** UI to configure multi-step workflows without coding, using conditional logic and loops.
- **SDK & API:** Expose secure API for third-party integrations and custom agent development.
- **Analytics & KPI dashboards:** Visualizations for business metrics (conversion, revenue, CSAT) and operator metrics. Provide cost tracking and forecasting.

7. Risks / Failure Modes and Mitigations

Risk/Failure Mode	Cause	Mitigation
Infinite loops / runaway costs	Agents call tools repeatedly or spawn excessive workers.	Set iteration limits and timeouts ¹⁵ ; enforce budget and rate limits in connectors; monitor cost anomalies and throttle tasks ¹⁰ .
Hallucinations & incorrect actions	LLM produces incorrect outputs; tasks executed on wrong data.	Use retrieval grounding and reasoning checks before actions ⁶ ; quality & safety critic validates outputs and business rules ¹³ ; high-risk actions require approval.
Security breaches & data leaks	Agents or tools expose sensitive data.	PII redaction and classification ²³ ; signed connectors with scopes ⁷ ; isolate high-risk agents in sandboxes ²⁴ ; audit logs for every action.
Lack of transparency / trust	Users cannot see what agents do.	Provide complete logging and audit trail ¹⁶ ; show chain-of-thought summaries and tool calls; highlight risk tiers and policies.
Over-engineering & complexity	Too many agents, features and orchestration patterns.	Follow Microsoft's guidance to use the simplest architecture needed ¹ ; start with single agents and sequential patterns; postpone optional features to Phase 2.

Risk/Failure Mode	Cause	Mitigation
Regulatory non-compliance	Agents operate outside data residency or HIPAA/GDPR rules.	Ensure connectors enforce data residency; maintain auditable records of prompts and data flows ²⁵ ; integrate policy engine for compliance checks.

8. KPI Dashboard Specification

The KPI dashboard (in the UI) should present both **operator metrics** and **business metrics**:

Metric Type	Example Metrics	Notes
Operator KPIs	Queue length, average wait time, average task duration, task success rate, escalation rate (tasks requiring human approval) ¹⁸ , consistency rate (similar inputs yield similar outputs), exploit exposure (# of successful jailbreaks) ¹⁸ , cost per task, tokens used, worker utilisation.	Real-time charts and moving averages; ability to filter by agent role and risk tier.
Business KPIs	Orders processed, revenue generated/refunded, customer satisfaction (CSAT), conversion rates, churn, time saved vs manual workflows, error rate on order fulfillment.	Derived from connectors (Shopify, CRM); align with e-commerce and personal-assistant use cases.
Safety KPIs	Number of blocked actions, policy violations, false positives/negatives, frequency of compliance exceptions.	Feed into policy tuning.

The dashboard should allow operators to drill down by period, agent and workflow; anomalies should trigger alerts (e.g., if refund errors spike).

9. Workflow Examples

Shopify Order Management

1. **User goal**: "Process today's Shopify orders and restock low-inventory items."
2. **Planner agent** decomposes the goal into tasks: fetch orders, analyze inventory, initiate restock purchase orders, and notify customers.
3. **Shopify agent** pulls orders via the Shopify connector and returns structured data. The Data Analyst generates a summary of inventory levels; if items fall below threshold, the agent composes a purchase order using a supplier API (under policy). The Planner checks risk tiers; purchase orders above a set amount require approval.
4. **Messaging agent** drafts emails to customers with shipping updates and sends them automatically for low-risk messages. Refunds above \$100 trigger the approval gate before execution ¹¹ .
5. **Quality & Safety critic** validates that all actions comply with business rules and schema; high-risk actions wait in the approval queue. Upon human approval, the orchestrator executes the actions.
6. **Outcome**: The dashboard shows tasks completed, revenue and restock orders; KPIs update accordingly.

Personal Assistant Workflow (Executive Assistant)

1. **User goal:** "Plan my next week: schedule meetings, prioritize tasks and summarize relevant news articles."
2. **Planner** breaks down tasks: gather existing calendar events, schedule new meetings with specified participants, compile a task list, and deliver a news summary.
3. **Calendar connector** fetches the user's availability. The Messaging agent sends meeting invites. Conflicting time slots trigger a suggestion list, which the user approves. The Researcher agent fetches and summarizes news relevant to the user's domain, ensuring retrieval is grounded in trusted sources.
4. **Data Analyst** compiles tasks into a spreadsheet and computes estimated time allocation. The Personal Assistant agent produces an agenda summary.
5. **Approval gate** is used only for scheduling meetings that require external participants or incur costs; otherwise actions run automatically.
6. **Outcome:** The UI displays scheduled meetings, the task list and news summary. The user can adjust any item before finalization.

10. What to Ship First (Fastest PMF Proof)

Recommendation: Prioritize a **transparent, Shopify-focused MVP** with built-in guardrails and clear UI. Early adopters in e-commerce (solo founders and small teams) will benefit the most from automating order management, inventory updates and customer communications. Integrate the personal assistant workflow to showcase generality and gather feedback. Provide reliable logs, approval flows and KPI dashboards; this builds trust and differentiates Empyralis from opaque competitors. Defer advanced group chat orchestration, complex workflow builders and dev-ops agents until the core experience proves useful and stable.

Build Checklist

Priority	Task
P0 – Core	Implement orchestrator with task queue, worker pool manager and memory store. Build Shopify, email, calendar and spreadsheet connectors with signed manifests. Develop planner, researcher, messaging, Shopify and critic agents. Create UI panels for task queue, task detail with approvals, and agent status. Add basic logs and audit trails. Implement input/output guardrails and policy engine for refunds and sensitive data. Ship personal assistant and Shopify workflows.
P1 – Enhancements	Add auto-scaling controller with queue-based scaling and limits. Implement Approval queue in UI. Extend observability plane with cost and performance metrics. Add Data Analyst and Finance agents. Support exportable logs and compliance reports. Optimize memory retrieval and implement caching.
P2 – Phase 2	Introduce DevOps agent and group chat orchestration; build workflow builder with conditional logic. Implement SLA-based scaling and predictive models. Enhance UI with customizable dashboards and analytics. Provide SDK/API for custom agent development. Add multi-approval patterns (two-key rule, time-delayed actions) for high-risk flows. Integrate reliability scoring and exploit exposure metrics.

Cut List (What Not to Build Now)

- **Unbounded multi-agent chat protocols** – group-chat or debate patterns increase complexity and resource usage ¹⁴; defer until Phase 2.
- **General-purpose code deployment (DevOps agent)** – not required for Shopify/assistant MVP; high risk and heavy integration.
- **Extensive third-party integrations** (e.g., Slack, HubSpot, Notion) – focus on a few high-impact connectors first.
- **Custom workflow builder / no-code editor** – adds significant UI complexity; postpone until core flows are proven.
- **Complex analytics & AI optimization engines** – start with basic KPI tracking; advanced predictive analytics can wait.
- **Full chain-of-thought display** – provide sanitized summaries; avoid exposing raw reasoning in the UI, both for IP protection and user simplicity.

By focusing on a lean architecture with strong guardrails, transparent UI and targeted workflows, Empyralis can deliver a valuable agent workspace within weeks, then expand systematically based on user feedback and performance metrics.

¹ ¹⁴ ¹⁵ ²¹ **AI Agent Orchestration Patterns - Azure Architecture Center | Microsoft Learn**
<https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns>

² ³ **Top 9 Open-source AI Agent Frameworks for 2025**
<https://www.graphbit.ai/resources/blogs/top-9-open-source-ai-agent-framework-2025/>

⁴ ⁵ ⁶ ⁸ ¹¹ ¹² ¹³ ¹⁷ ¹⁹ ²⁰ ²³ **Design Patterns For Safe Agentic AI: Guardrails, Policies And Human Approval Flows - Rahul kolekar**
<https://rahulkolekar.com/design-patterns-for-safe-agentic-ai-guardrails-policies-and-human-approval-flows/>

⁷ ⁹ ¹⁰ ¹⁶ ¹⁸ ²² ²⁴ ²⁵ **From Chaos to Clarity: Establishing Transparency & Accountability in AI Agent Deployments**
<https://www.2tolead.com/insights/from-chaos-to-clarity-establishing-transparency-accountability-in-ai-agent-deployments>